

RAMIFY OS v11.0.6

FastAPI/Python David Feedback Coverage, Deep-Audit and Regression Report

Prepared 8 September 2026

Release result 379 tests passed, 4 optional tests skipped, 1,312 subtests passed, 0 failures.

Scope: this report covers the FastAPI/Python demonstration and the David feedback dated 26, 27 and 28 August 2026. The separate Go feedback is outside this build scope.

The audit focused on deterministic resolution, evidence/claim binding, actor-policy safety, transaction authority, receipt and action-ledger verification, human successor records, optional AI adapter guardrails, Proof Pack portability, release packaging and final-presentation evidence.

1. Final regression evidence

Item	Result
Tests	379 passed
Optional tests	4 skipped - including browser E2E in locked-down Chromium environments
Subtests	1,312 passed
Failures	0
Test runner	pytest
Command	PYTHONPATH=backend pytest -q
Static Python check	compileall passed before release cleanup
Frontend JavaScript syntax	node --check passed
Release package audit	release_check.py passed after removing caches

2. David feedback coverage

Area	Coverage in this build
Evidence-derived result	Evidence records store underlying facts. RATIFY derives validity states; no precomputed evidence_expired answer is used.
Evidence integrity	RATIFY opens shipped artefacts, recomputes SHA-256, verifies issuer Ed25519 signatures and checks signed subject/issuer/date/scope binding.
Claim consistency	Claim verdicts reuse the evidence-validity/integrity evaluator, so revoked, not-yet-valid and malformed evidence cannot be displayed as accepted.
Policy precedence	Policy pack wording matches the engine: all applicable rules are evaluated, the most restrictive posture determines the result, and precedence chooses the primary reason.
Brightway chronology	The impossible retrieved-before-issued evidence date is corrected and provenance chronology is checked.
Actor policy	Objective product posture is completed first; actor policy can only narrow and classifies rules as commercial or trust-derived.
Agent profiles	Profiles expose constrained inputs only; persisted JSON is revalidated on load and fails closed if malformed.
Primitive order	Docs, technical page and presentation use identify -> resolve -> status -> verify -> assess.
Proof Packs	Seller-risk uses Covelane N95; substitution uses Stonefield Zinc Gluconate. Quick 5-scenario and Extended 6-receipt packs are explicitly separate.
Alternatives	Alternatives are only offered for objectively acceptable products blocked by commercial policy; exploratory suggestions carry no purchase-authority receipt ref.

Area	Coverage in this build
PydanticAI	Optional comparison adapter is constrained to local loopback Ollama, supplied candidate refs and clamped confidence. It remains outside trust authority.
Freshness terminology	User-facing wording separates evidence validity, projection freshness and purchase-authority validity window.
Security scope	Docs state real hash/signature mechanics over synthetic data; no production key custody, immutable ledger or real certification is claimed.
Presentation	Deck text is updated to current v4 evidence and includes screenshot-based evidence slides rather than only repeated claims.

3. Transaction and cryptographic hardening

Area	Verified behaviour
Basket binding	Checkout re-reads each signed decision receipt and compares subject, product, quantity, unit price, total, actor, posture, decision, human-authorisation and successor linkage. Manual cart edits fail closed.
Order/requisition verification	Transaction records re-verify linked decision receipts cryptographically and compare transaction-critical fields; they do not trust only a quoted hash.
Receipt semantics	Record integrity is separate from purchase-authority validity. An old intact receipt remains authentic history but cannot authorise a new purchase.
Action ledger	Hash-chain verification validates receipt reference, receipt hash and linked receipt integrity.
Human successor actions	Generic Action Gate accepts only actions actually authorised by the signed successor receipt; original machine receipts remain unchanged.
Signer lifecycle	Valid legacy signer keys migrate by deriving or repairing the public companion. Corrupt private signer state fails closed.
Quantity boundary	The domain engine rejects non-integer, zero, negative and excessive quantities; safety does not rely only on HTTP validation.
Issuer reference wording	Receipt verification now reports issuer_refs_known. The old issuer_chain_valid alias remains only for backward compatibility.

4. Manual and optional browser validation

Flow	Result
Consumer API flow	Assess Apex -> signed receipt -> Add to basket -> checkout -> signed order verification passed.
Human procurement API flow	Northbeam procurement hold -> human override successor -> Action Gate -> signed requisition verification passed.
Action ledger	Ledger verification passed after manual smoke flows.
Quick Proof Pack	Generated ZIP extracted and standalone verifier returned success for all included receipts.
LLM dependency	Deterministic assessment and transaction flow completed with RAMIFY_DISABLE_AGENT=1.
Browser E2E tests	Optional Playwright/Chromium tests are included for consumer checkout and human-review successor flow. In this locked-down QA runner Chromium is blocked by host policy, so the tests skip cleanly.

5. Performance sample

A clean Apex assessment was executed 100 times after 10 warm-up runs in the QA container. This is a small-dataset engineering sample, not a production or presentation-laptop benchmark.

Metric	Milliseconds
Median	4.452
P95	8.802
Maximum	11.607

6. Final submission evidence still required from the team repository

The extracted ZIP cannot honestly provide the final Git commit SHA or clean/dirty state. Run scripts/capture_release_evidence.py from the final team Git checkout after the release commit. It captures commit, branch, commit date, clean/dirty state, timestamp, exact test command and machine-readable output.

7. Release conclusion

The FastAPI build now covers David's actionable feedback points at code, data, proof-pack, terminology, testing and presentation level. The remaining responsibility is final team-side evidence capture from the actual Git repository and manual browser demonstration on the presentation laptop.